Unit 4
Data and Analysis

From raw facts to a relational database you can query

DATA VS INFORMATION | KEYS & INTEGRITY | 1:1 · 1:M · M:N | ER MODELS | MS ACCESS OBJECTS

A month of temperatures is data. The average and the heatwave week are information.

0. Data is not yet knowledge

Data are unprocessed facts — numbers, text, images — quantitative (countable/measurable) or qualitative (descriptive). **Information** is data after organising, processing, and interpreting. A **database** is an organised, stored collection that a program can retrieve without hunting through notebooks.

$$\text{Data} \xrightarrow{\text{processing}} \text{Information} \xrightarrow{\text{organisation}} \text{Database}$$

Raw: $80, 75, 90, 70, 85$. Information: class average $= 80\%$.

Table 1. Data versus information

	Data	Information
Definition	Raw facts	Processed, meaningful
Example	$85, 90, 75$	Average marks $= 83.3\%$
Usefulness	Low by itself	Supports a decision
Form	Numbers, text, images, video	Reports, summaries, charts

1. DBMS — the software that looks after the boxes

A **Database Management System** (MS Access, MySQL, Oracle, SQL Server) stores, protects, and retrieves data. Benefits: less redundancy, consistent copies, security, integrity constraints.

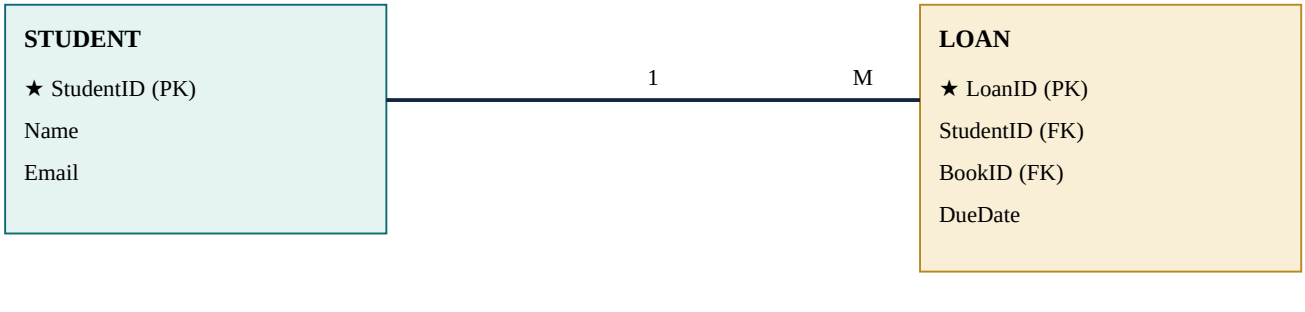
TABLE
A grid of related data. One subject (Students, Books).

RECORD / ROW / TUPLE
One instance (one student).

FIELD / COLUMN / ATTRIBUTE
One property (Name, Age, Email).

2. Keys and relationships — how tables refuse to become spaghetti

A **candidate key** is any field (or minimal set) that uniquely identifies a record. You promote one candidate to **primary key (PK)**. The unused candidates become **alternate keys**. A **foreign key (FK)** in table B stores a PK value from table A so the two rows belong together.



One student may have many loans. A loan belongs to exactly one student. This is 1:M.

Figure 1. Primary key on the “one” side, foreign key on the “many” side.

Table 2. The three relationship cardinalities

Type	Meaning	How we store it
$1:1$	Each A matches at most one B	PK of A also stored (uniquely) in B
$1:M$	One A, many B — the common case	FK of A sits in B
$M:N$	Many A related to many B	Break with a **junction / bridge** table that holds both FKs

Students and classes are $M:N$ (a student takes many classes; a class has many students). The bridge `Enrolment(StudentID, ClassID)` turns that into two $1:M$ links.

2.1 Referential integrity

A foreign key value must already exist as a primary key, or be empty if the design allows it. You cannot loan a book that is not in BOOKS. **Cascade update** rewrites child FKs when a parent PK changes. **Cascade delete** removes child rows when the parent dies — use it only when those children have no meaning alone (loan rows without a student).

3. Entity-Relationship modelling

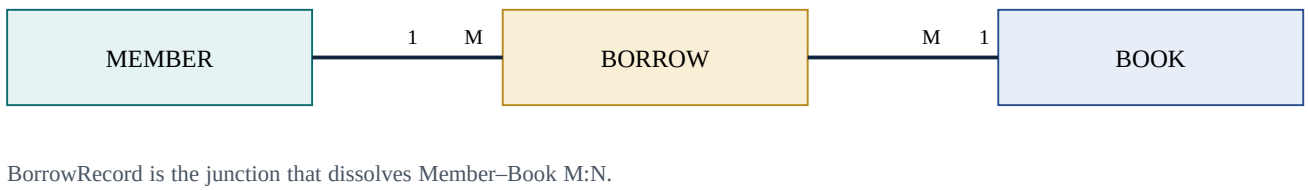
An **entity** is a real-world thing we store (Book, Member). An **attribute** is a property. A **relationship** is a verb between entities (Member *borrows* Book). Draw rectangles for entities, ovals or listed fields for attributes, diamonds or labelled lines for relationships, and mark PKs.

3.1 Library case — from story to schema

A library lends many books to many members. Direct $M:N$ is illegal in a pure relational schema, so we introduce **BorrowRecord**.

$$\begin{aligned}&\text{Book}(\underline{\text{BookID}}, \text{Title}, \text{Author}, \text{Publisher})\\&\text{Member}(\underline{\text{MemberID}}, \text{Name}, \text{ContactInfo})\\&\text{BorrowRecord}(\underline{\text{BorrowID}}, \text{MemberID}, \text{BookID}, \text{BorrowDate}, \text{ReturnDate})\end{aligned}$$

Underlined fields are primary keys. `MemberID` and `BookID` in BorrowRecord are foreign keys.



BorrowRecord is the junction that dissolves Member–Book M:N.

Figure 2. ER sketch of the library. Practise this exact triangle; it is the course's running example.

4. Database objects in MS Access

Access (and most desktop RDBMS tools) expose four objects students must name:

TABLES
Store the data. Created in Datasheet View (spreadsheet-quick, weak control) or Design View (name every field, pick a data type, set the PK).

FORMS
Friendly data-entry windows. Form Wizard can build a student registration screen on top of a table.

QUERIES
Questions: filter `Department = "BCA"`, sort, pick columns, group, average.

REPORTS
Printable summaries and charts from selected data — what a principal actually reads.

Table 3. Typical Student table in Design View

Field	Data type	Notes
StudentID	AutoNumber	Primary key
Name	Short Text	Full name
Department	Short Text	BCA, BCOM, …
Age	Number	Years
Email	Short Text	Contact

LAB DRILL Create `StudentDB`. Build the table in Design View (not Datasheet). Mark StudentID as PK (key icon). Enter four sample rows. Build a form with the wizard. Write a query that lists only BCA students sorted by Name. Create a report of that query.

Self-check

1. Marks $70, 80, 90$ are data. Write one sentence of information.
2. Why is $M : N$ stored with a third table?
3. Cascade delete on Member → BorrowRecord: what happens if we delete a member who still has open loans?

Answers. (1) e.g. "The mean mark is 80, so the class met the pass threshold." (2) A relational table cannot hold a repeating group of other keys cleanly; the junction table stores each pair once. (3) Those loan rows disappear too — which may be wrong if you still need history. Prefer restrict/cascade-update unless history is disposable.

Recap

1. Data → information → database. DBMS reduces redundancy and enforces rules.
2. Table / record / field. PK unique; FK points at a PK.
3. $1:1$, $1:M$, $M:N$ (junction). Referential integrity + optional cascades.
4. ER: entities, attributes, relationships. Library: Book, Member, BorrowRecord.
5. Access objects: tables, forms, queries, reports. Prefer Design View for real tables.

Draw Figure 2 from memory, then name every key.